

ROS2 day3 hw1 레포트

로봇 예비단원 주호진

1. 과제

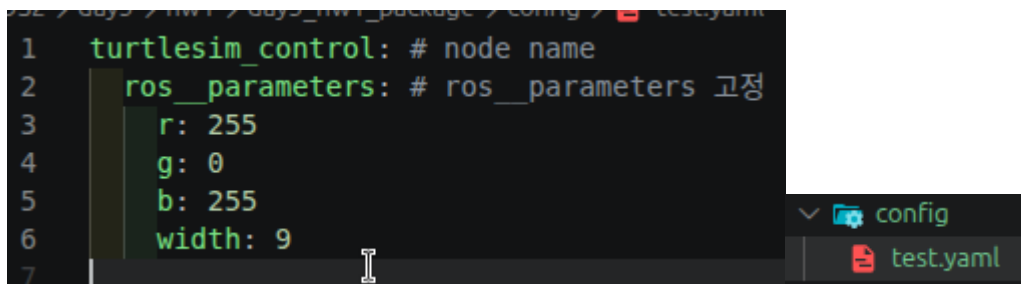
ros2 1일차 과제3번 패키지에서 굵기, 색상을 파라미터 설정, yaml파일에서 불러오는 코드 추가

-> 작동 영상 첨부 (파라미터 변경 전·후 과정을 한 영상에 포함)

2. 코드

ros2 1일차 과제 3번을 py로 짰 경우 AI를 사용해서 cpp 코드로 변환하도 된다고 하셔서 AI로 변환한 코드를 넣고 수정한 부분부터 기재했다.

```
1 turtlesim_control: # node name
2   ros__parameters: # ros__parameters 고정
3     r: 255
4     g: 0
5     b: 255
6     width: 9
```



Yaml 파일은 기존에 setPen에 있던 파라미터들을 넣어서 다음과 같이 설정했다.

Yaml 파일의 디렉터리는 config 하위에 두는 것이라고 하여 config 폴더를 생성하고 그 하위에 위치해냈다. Yaml 파일의 이름은 어떤 내용이 들었는지 알아보기 쉽게 짓는데, 일단 처음 해보는 거라 test라고 지었다.

- 파라미터 받아오기

```

from launch import LaunchDescription
from launch.substitutions import PathJoinSubstitution
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare

def generate_launch_description():
    # 패키지 내 config/test.yaml 경로를 상대 참조로 생성
    yaml_file_path = PathJoinSubstitution([
        FindPackageShare('day3_hw1_package'),
        'config',
        'test.yaml'
    ])

    return LaunchDescription([
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            name='turtlesim_node'
        ),
        Node(
            package='day3_hw1_package',
            executable='day3_hw1_package', # cmakeLists에 나와있는 노드 이름
            name='turtlesim_control',
            parameters=[yaml_file_path] # 파라미터 파일 전달
        )
    ])

```

Launch 파일을 만들고 turtlesim과 기존에 있던 node를 설정해준다. Launch 시 yaml 파라미터를 설정한 노드로 전달해준다.

```

3  #include "rclcpp/rclcpp.hpp"
4
5  class Parameter
6  {
7  public:
8      explicit Parameter(rclcpp::Node *node);
9
10     int r() const
11     {
12         return r_;
13     }
14     int g() const
15     {
16         return g_;
17     }
18     int b() const
19     {
20         return b_;
21     }
22     int width() const
23     {
24         return width_;
25     }
26
27 private:
28     rclcpp::Node *node_;
29     int r_, g_, b_, width_;
30 };

```

파라미터를 읽어주는 parameter.hpp, parameter.cpp를 만들어준다.

```

1  #include "day3_hw1_package/parameter.hpp"
2
3  Parameter::Parameter(rclcpp::Node *node) : node_(node)
4  {
5      node_>declare_parameter<int>("r", 0);
6      node_>declare_parameter<int>("g", 0);
7      node_>declare_parameter<int>("b", 0);
8      node_>declare_parameter<int>("width", 1);
9
10     r_ = node_>get_parameter("r").as_int();
11     g_ = node_>get_parameter("g").as_int();
12     b_ = node_>get_parameter("b").as_int();
13     width_ = node_>get_parameter("width").as_int();
14 }

```

Parameter 생성자는 파라미터의 기본값을 설정한 뒤 런치 파일이 node에 지정해둔 yaml 값을 읽어온다.

```
std::unique_ptr<Parameter> config_;
};
void setPen(std::uint8_t red, std::uint8_t green, std::uint8_t blue, std::uint8_t width, std::uint8_t off = 0);
```

Rgb, width를 가진 parameter 객체를 node.hpp에 선언하고, 기존에 setPen에 직접 입력해주던 매개변수는 지워준다.

```
TurtlesimControl::TurtlesimControl()
{
    config_ = std::make_unique<Parameter>(this);
}
```

Node가 생성될 때 parameter도 생성(=config_)해준다. 이때 위에 설명한 파라미터 생성자가 파라미터 값을 읽는다.

```
void TurtlesimControl::setPen(std::uint8_t off)
{
    if (!pen_client_>wait_for_service(100ms))
        return;

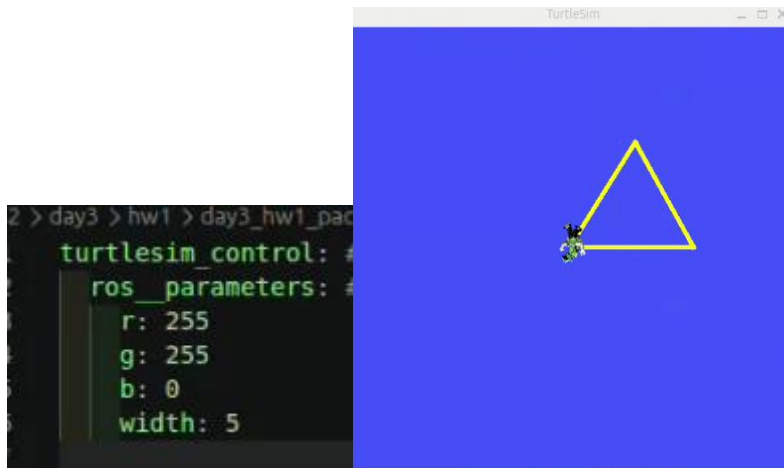
    auto request = std::make_shared<turtlesim::srv::SetPen::Request>();
    request->r = config_>r();
    request->g = config_>g();
    request->b = config_>b();
    request->width = config_>width();
    request->off = off;
    pen_client_>async_send_request(request);
}
```

직접 지정해주던 값 대신에 parameter를 통해 읽어온 값을 지정해준다.

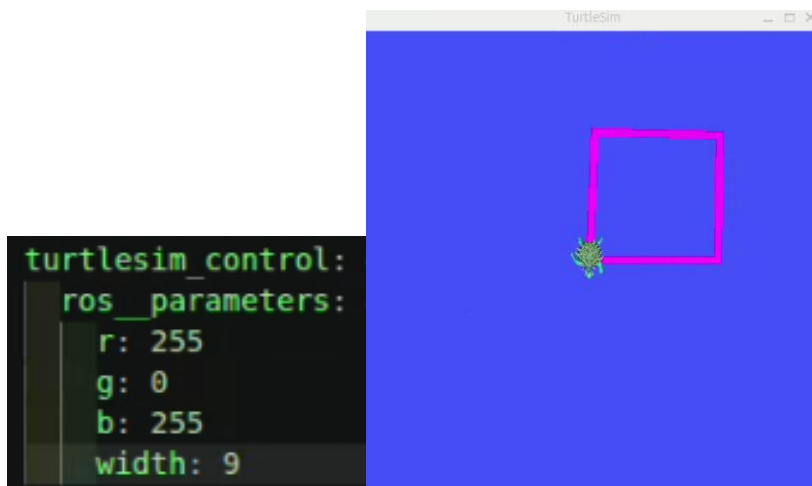
3. 작동 영상

[작동 영상](#)

R, g 255, width 5일 때



R, b 255, width 9일 때



4. AI 보고서

- Yaml, launch 관련

<https://share.gemini.google/NCLKKE5DTYHk>

ros2 1일차 과제3번 패키지에서 굵기, 색상을 파라미터 설정,
yaml파일에서 불러오는 코드 추가

이게 과제고 ros2 1일차 3번 과제가 노드 토픽으로 터틀봇
움직여서 도형그리기였거든? 이게 과제가 해석이 안되는데
패키지에서 설정한 값들을 파라미터 설정하라는 게 무슨 말이야?

기존 과제(터틀봇 도형 그리기)에서 하드코딩했던 펜의 색상(R, G, B)과 굵기(width) 값을 소스코드 수정 없이 외부에서 바꿀 수 있도록 ROS2 파라미터(Parameter)로 만들고, 이를 **YAML 파일**을 통해 불러오라는 뜻입니다.

Yaml 개념을 처음 접해서 과제가 한줄인데 이해가 안가서 질문했다. 좀 찾아보면서 아래 문장과 같이 이해하였다.

ros2 1일차 과제3번 패키지에서 굵기, 색상을 (yaml파일을 통해) 파라미터 설정, yaml파일에서 (파라미터를) 불러오는 코드 추가

파라미터를 선언할 때 그 값은 야멜 파일에 100이라 적혀있으면
코드에도 100이라고 적는거야?

아니요, 코드와 YAML 파일에 같은 값을 똑같이 적을 필요는 없습니다. 코드에 적는 값은 YAML 파일이 없을 때 사용되는 기본값(Default Value)입니다.

ex)

yaml 파일에 저장한 파라미터가

```
update_rate: 10
speed: 1.0
```

와 같다면 다음과 같이 **Parameter** 선언 및 **get parameter** 부분을 소스 코드에 추가해야 함

```
this->declare_parameter("update_rate", 10);
this->declare_parameter("speed", 1.0);

int update_rate = this->get_parameter("update_rate").as_int();
float speed = this->get_parameter("speed").as_double();
```

처음에 ppt에 있는 예제를 보고 yaml에 적힌 값을 코드에 똑같이 옮겨적어야 하는 걸로 이해한 뒤에 했던 질문이다. 똑같은 값을 적는데 왜 yaml 파일로 관리하지? 라는 생각이 들어 질문을 했는데 코드에 적는 값은 yaml파일에서 해당 파라미터가 없을 때 사용하는 default값을 적는 것임을 알았다.

yaml 파일의 파라미터 값을 그냥 코드 에디터에서 수정하면 바로 적용되는거야?

Yaml 파일이 수정되면 바로 적용되는 건지 물어봤다. 빌드를 다시 해야 적용된다.

/turtle1/cmd_vel [geometry_msgs/msg/Twist] 메시지를 yaml로 어떻게 작성하는거야

/turtle1/cmd_vel 은 파라미터가 아니라 거북이에게 속도 명령을 보내는 토픽(Topic)이므로 YAML 파일에 작성하는 대상이 아닙니다.

YAML 파일에는 과제 요구사항인 펜의 색상(R, G, B)과 굵기(width) 같은 파라미터를 아래와 같은 형식으로 작성해야 합니다.

YAML

```
# 본인의 파이썬 노드 이름(executable 또는 node name)
turtle_drawing_node:
  ros__parameters:
    pen_r: 255
    pen_g: 100
    pen_b: 0
    pen_width: 5
```

ros__parameters는 고정이야?

turtlesim/srv/SetPen 이거잖아

네, `ros__parameters` 는 고정된 키워드입니다!

ROS2에서 YAML 파일을 읽어올 때, 파라미터 블록의 시작을 알리는 약속된 이름이기 때문에 반드시 `__` 글자 그대로(언더바 두 개 포함) 적어야 합니다. 다른 이름으로 바꾸면 ROS2가 파라미터로 인식하지 못합니다.

ros2에서 사용하는 yaml 형식에 대해 질문했다. Ros__parameters라고 적혀있어야 ros2에서 인식한다고 한다.

파라미터 커스텀을 msg에 맞게 yaml로 짜고 그걸 호출해서 사용한다는거지

정확히 이해하셨습니다! 다만 한 가지 용어를 살짝 짚고 넘어가자면, 토픽에서 사용하는 `msg` 가 아니라 `turtlesim/srv/SetPen` 서비스의 요청(Request) 필드(`r`, `g`, `b`, `width`, `off`)에 맞게 파라미터 이름을 정하고 YAML 파일을 작성하는 것입니다.

yaml 파일의 이름은 아무렇게나 해도 상관없는거지

네, YAML 파일 이름은 원하시는 대로 **아무렇게나** 지어도 상관없습니다 (`config.yaml` , `turtle_param.yaml` , `setting.yaml` 등).

yaml파일은 보통 어떤 폴더 하위야



ROS2 패키지에서 YAML 파라미터 파일은 보통 패키지 루트 디렉토리 아래에 만드는 `config` 폴더에 저장합니다.

○ 표준 ROS2 패키지 디렉토리 구조 예시:

- `my_package/` (패키지 폴더)
 - `config/` → `param.yaml` 같은 설정 파일들이 위치하는 곳
 - `launch/` → `turtle_launch.py` 같은 런치 파일들이 위치하는 곳

yaml에 관한 개념 정리와 yaml 파일을 어디 디렉터리에 관리해야 하는지 물었다. 국룰의 위치가 있지 않을까 싶어서 질문했고, config가 국룰이라고 해서 config에 두었다.

ros2 py 런치파일 작성하는 방법 자료 링크좀

[ROS2 공식 튜토리얼 문서\(Creating a launch file\)](#)

파이썬 런치 파일 기본 구조 예시 (`launch/turtle_launch.py`)

[XML](#)[YAML](#)[Python](#)

Copy and paste the complete code into the `launch/turtlesim_mimic_launch.py` file:

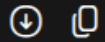
```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
            namespace='turtlesim1',
            executable='turtlesim_node',
            name='sim',
            arguments=['--ros-args', '--log-level', 'info']
        ),
        Node(
            package='turtlesim',
            namespace='turtlesim2',
            executable='turtlesim_node',
            name='sim',
            ros_arguments=['--log-level', 'warn']
        ),
        Node(
            package='turtlesim',
            executable='mimic',
            name='mimic',
            remappings=[
                ('/input/pose', '/turtlesim1/turtle1/pose'),
                ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel'),
            ]
        )
    ])
])
```

.py launch파일 정보가 있는 링크를 찾아달라고 질문하였다.

파일 경로를 packageShare로 상대참조 하라는데 관련 라이브러리좀 알려주 ㅊ

Python



```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare
from launch.substitutions import PathJoinSubstitution

def generate_launch_description():
    # 패키지 내 config/param.yaml 경로를 상대 참조로 생성
    yaml_file_path = PathJoinSubstitution([
        FindPackageShare('내패키지이름'),
        'config',
        'param.yaml'
    ])

    return LaunchDescription([
        Node(
            package='내패키지이름',
            executable='실행파일이름',
            name='turtle_drawing_node',
            parameters=[yaml_file_path] # 파라미터 파일 전달
        )
    ])

```

launch파일에서 절대로 "절대 경로"를 사용하지말고 가급적 package share를 이용해서 상대경로로 참조하라고 말씀해주셔서 package share 관련 라이브러리를 물어봐서 적용했다.

- 워크스페이스(ws) 관련

<https://share.gemini.google/6uAYXKbCzLLm>

지금 workspace src안에 패키지마다 심볼릭 링크를 걸어주고 있는데 package가 여러개 분류되어서 들어있는 최상위 폴더를 링크걸어줘도 인식하나?



네, 최상위 폴더를 심볼릭 링크로 걸어두어도 빌드 시스템이 하위 폴더들을 재귀적으로 탐색하여 패키지로 인식합니다.

```
hojin@hojins:~/colcon_ws/src/ROS2$ tree
```

```
.  
├── day1  
└── hw1
```

이 ROS2 폴더 안에는 컴파일 결과물이나 캐시 파일(`build` , `install` , `log` , `CMakeCache.txt` 등)은 전혀 없고, 빌드 설정 파일들만 포함되어 있습니다.

기존에 워크스페이스 src에 패키지마다 링크를 연결해서 사용하고 있었는데, 과제 제출 폴더를 통째로 git clone해서 사용해도 문제 없다고 말씀해주셔서 바꿨다.

처음에는 과제 제출 파일 전부를 링크를 걸었는데 ros2만 빌드하는 거니까 ros2 폴더만 심볼릭 링크로 걸어서 관리하도록 변경했다.